



First Meetup @ SF

Oct 5th, 2023

@ a16z San Francisco Office



About us



Woosuk Kwon
PhD Student, UC Berkeley

✕ [@woosuk_k](#)

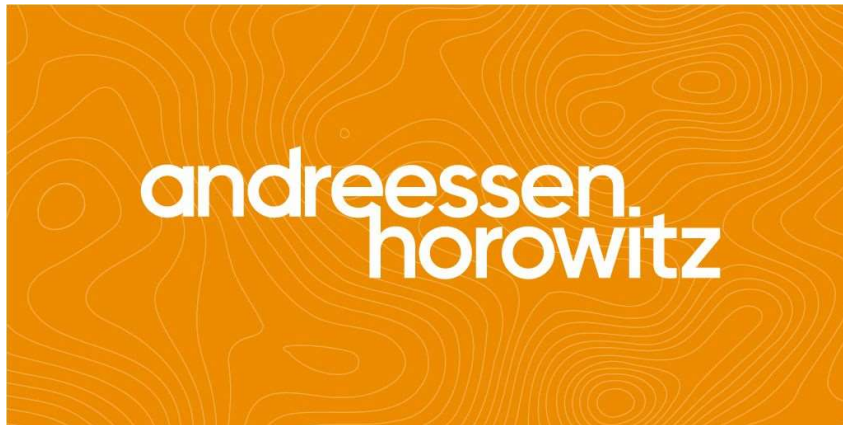


Zhuohan Li
PhD Student, UC Berkeley

✕ [@zhuohan123](#)



Thanks our sponsors!



for venue & open-source AI grant



for catering & contributions to vLLM

vLLM Overview

Woosuk Kwon

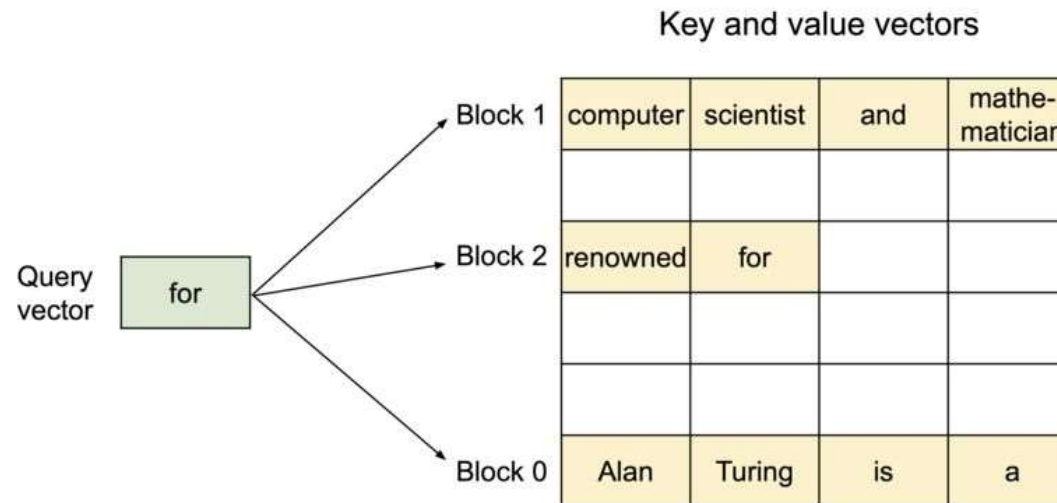


Our Goal

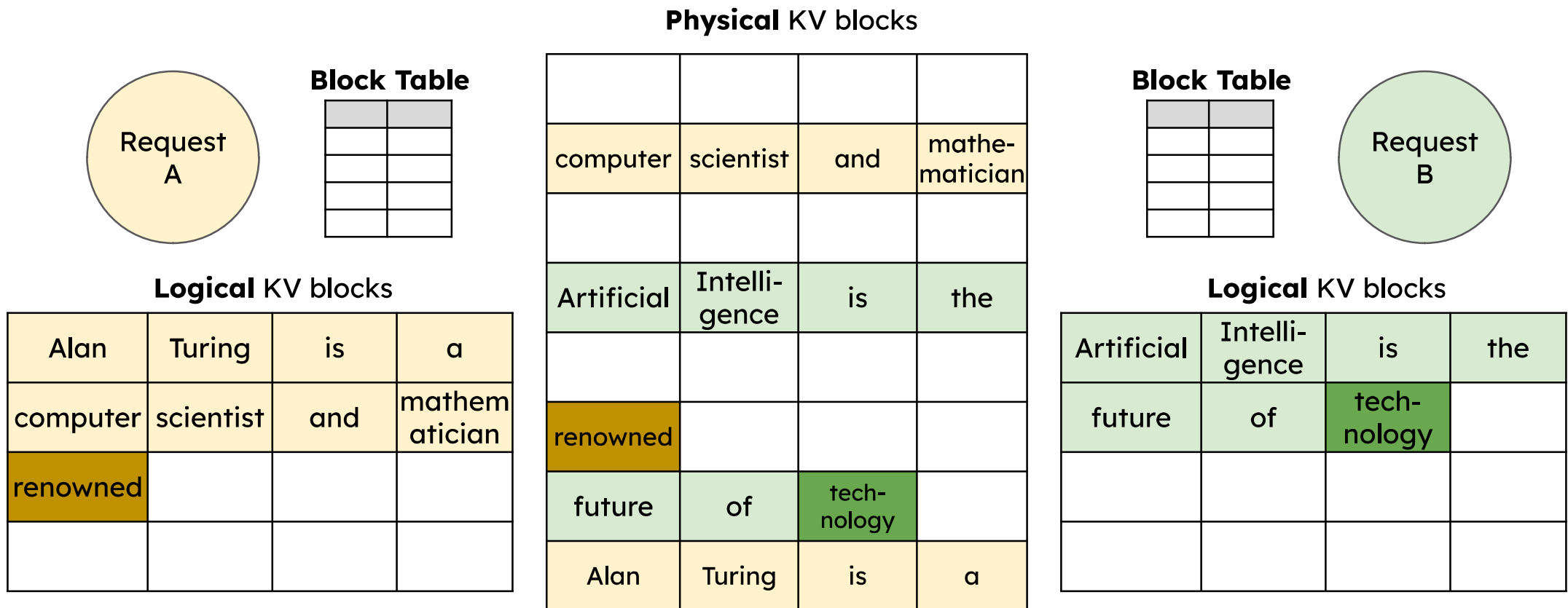
Build the **fastest** and
easiest-to-use open-source
LLM inference & serving engine

PagedAttention

An attention algorithm that allows for storing continuous keys and values in non-contiguous memory space.



Manage KV cache like OS virtual memory



vLLM Github Repo



<https://github.com/vllm-project/vllm>

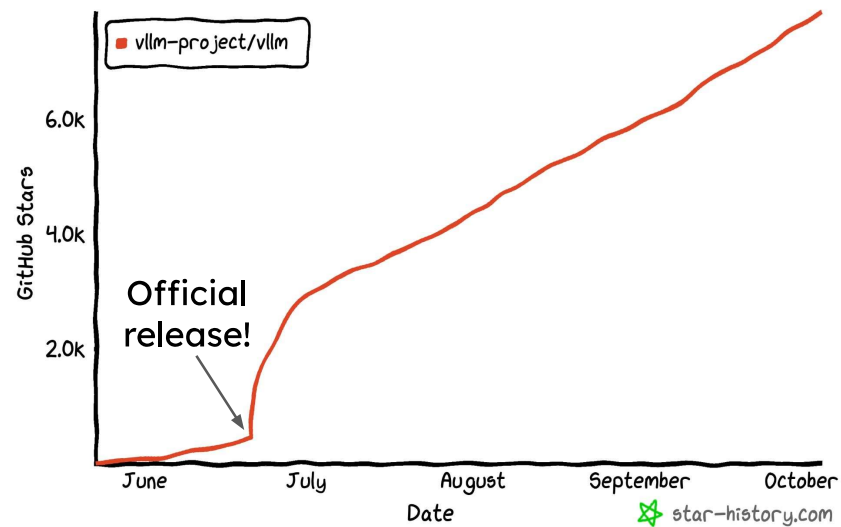


```
$ pip install vllm
```



7.9K Stars

v Star History



vLLM API (1): LLM class

A Python interface for offline batched inference

```
from vllm import LLM

# Example prompts.
prompts = ["Hello, my name is", "The capital of France is"]
# Create an LLM with HF model name.
llm = LLM(model="meta-llama/Llama-2-7b-hf")
# Generate texts from the prompts.
outputs = llm.generate(prompts)
```

vLLM API (2): OpenAI-compatible server

A FastAPI-based server for online serving

Server

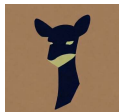
```
$ python -m vllm.entrypoints.openai.api_server --model meta-llama/Llama-2-7b-hf
```

Client

```
$ curl http://localhost:8000/v1/completions \
  -H "Content-Type: application/json" \
  -d '{
    "model": "meta-llama/Llama-2-7b-hf",
    "prompt": "San Francisco is a",
    "max_tokens": 7,
    "temperature": 0
  }'
```

LLM Adopters

Open-Source Projects



Im-sys/FastChat



OPENCHAT

Ai2 allenai/open-instruct



SkyPilot



WizardLM

...

Companies



anyscale



databricks



Meta

Replicate



Google Cloud



Modal

scale

...

..

BentoML



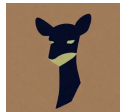
Lepton AI



nVIDIA

Uber

“Talk is cheap. Show me the code.”



lm-sys/FastChat

https://github.com/lm-sys/FastChat/blob/main/fastchat/serve/vllm_worker.py



OPENCHAT

https://github.com/imoneoi/openchat/blob/master/ochat/evaluation/run_eval.py

Ai2 allenai/open-instruct

https://github.com/allenai/open-instruct/blob/main/eval/toxigen/run_eval.py

SkyPilot

<https://github.com/skypilot-org/skypilot/blob/master/llm/vllm/serve.yaml>



WizardLM

https://github.com/nlpxucan/WizardLM/blob/main/WizardCoder/src/humaneval_gen_vllm.py



anyscale

https://github.com/ray-project/ray/blob/master/doc/source/serve/doc_code/vllm_example.py



BentoML

https://github.com/bentoml/OpenLLM/blob/main/openllm-python/src/openllm/models/auto/modeling_vllm_auto.py



Google Cloud

https://github.com/GoogleCloudPlatform/vertex-ai-samples/tree/main/community-content/vertex_vision_model_garden/model_oss/vllm



Lepton AI

<https://github.com/leptonai/examples/tree/main/advanced/vllm>

Meta

<https://github.com/facebookresearch/lama-recipes/tree/main/examples/vllm>



NVIDIA

https://github.com/triton-inference-server/tutorials/tree/main/Quick_Deploy/vLLM



Modal

https://github.com/modal-labs/modal-examples/blob/main/06_gpu_and_ml/vllm_inference.py

Replicate

<https://github.com/replicate/vllm-with-loras>

scale

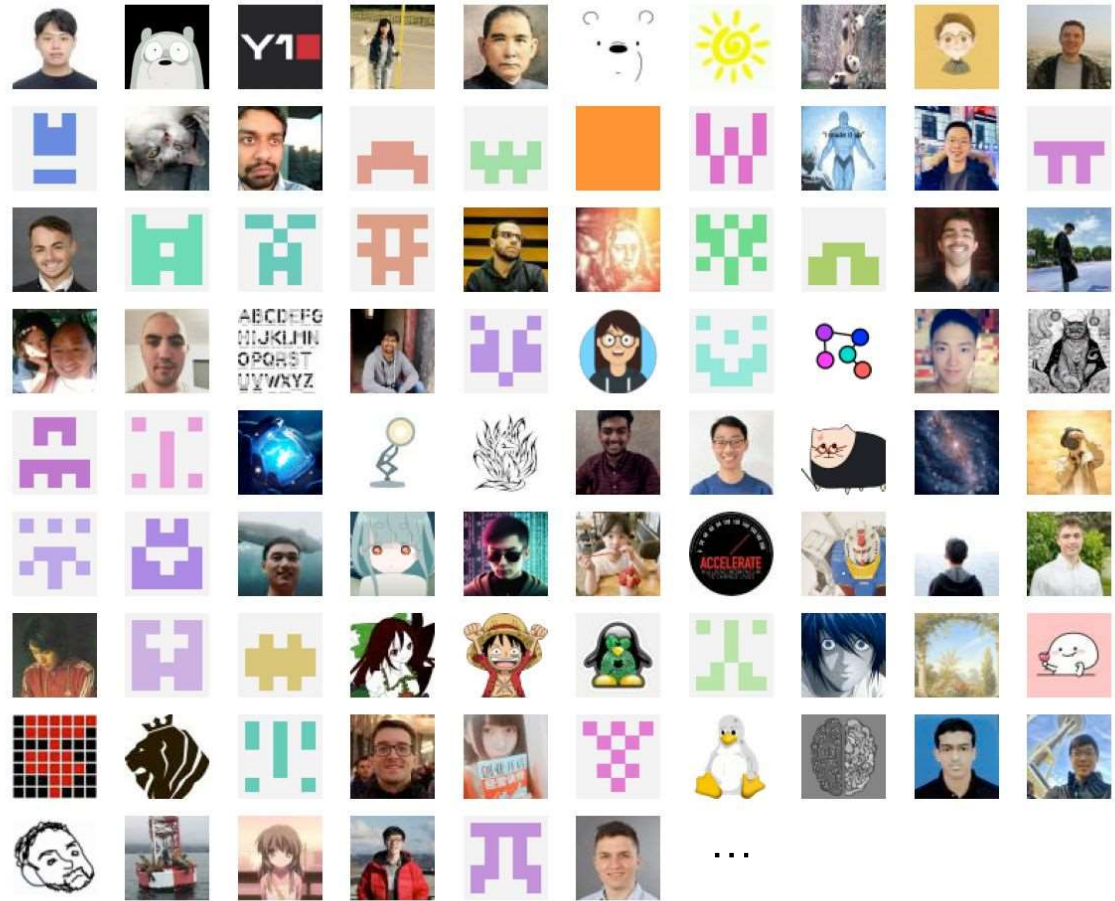
https://github.com/scaleapi/llm-engine/tree/main/model-engine/model_engine_server/inference/vllm

...

...

Contributors

Thanks to all the **contributors** who raised issues, participated in discussions, and submitted PRs!



vLLM System Walkthrough

Zhuohan Li

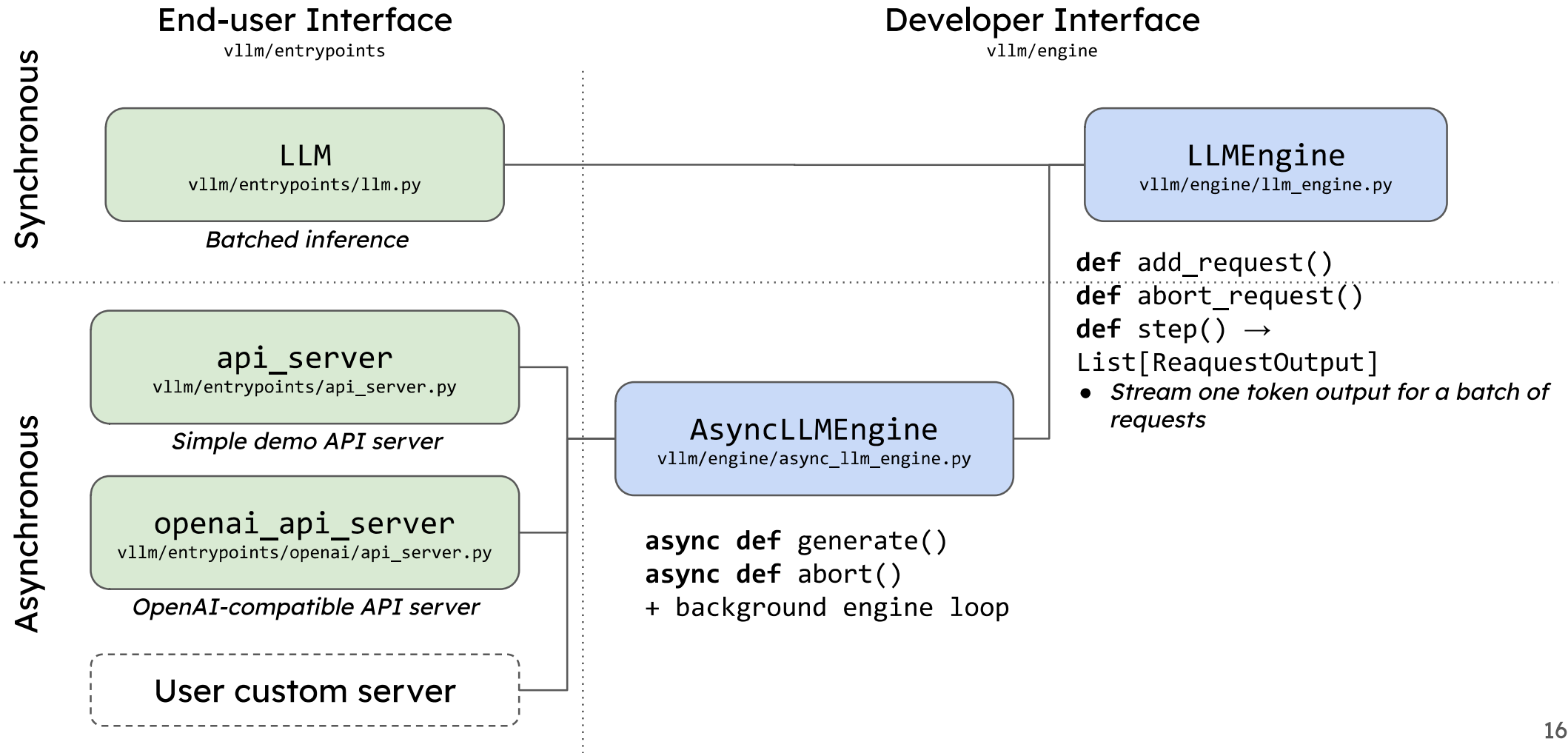


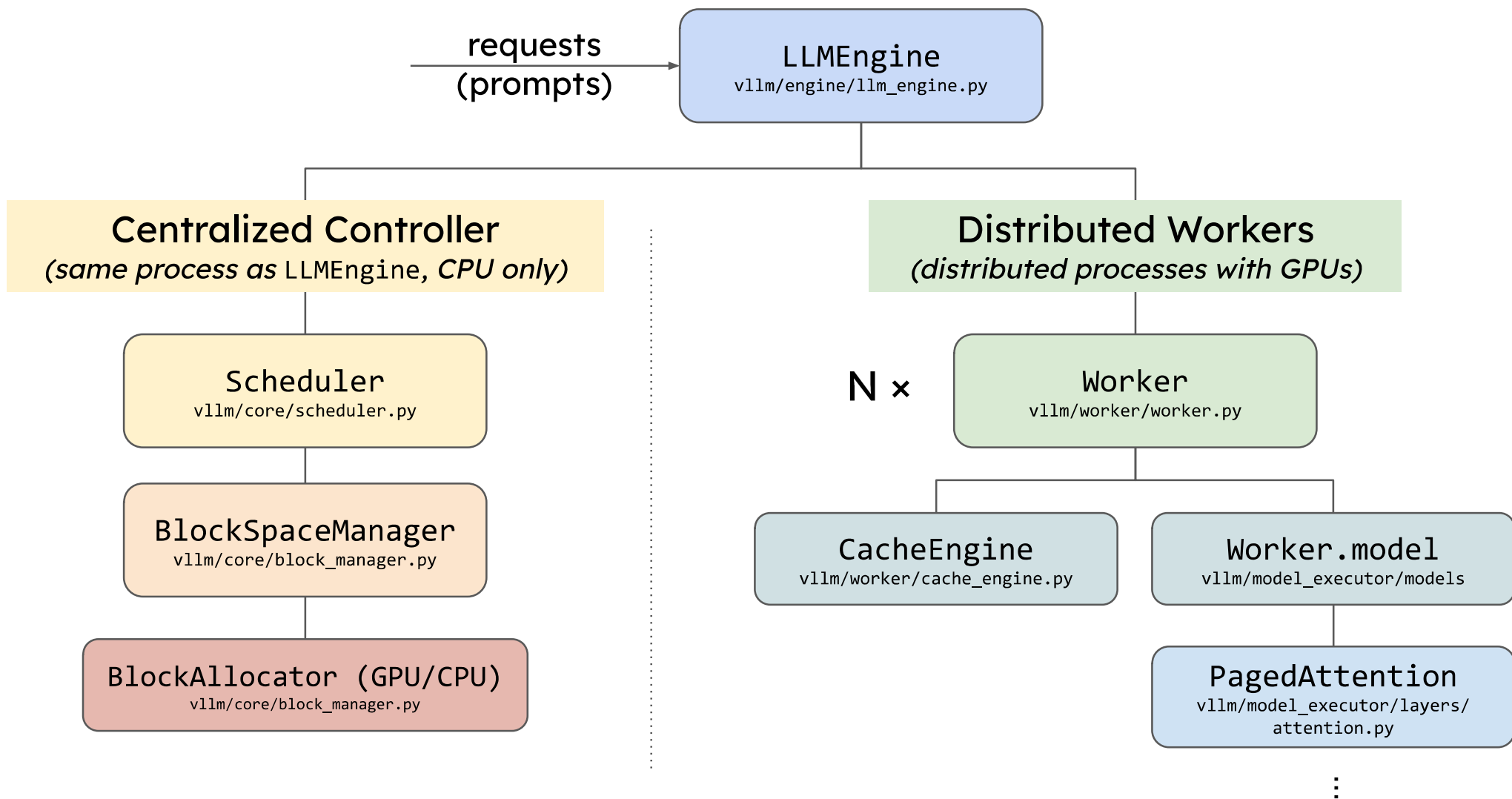
Goal of the walkthrough

1. Understand how vLLM processes a request and generates its outputs. Background for later content.
2. Learn where to modify if you would like to make a specific modification/contribution.



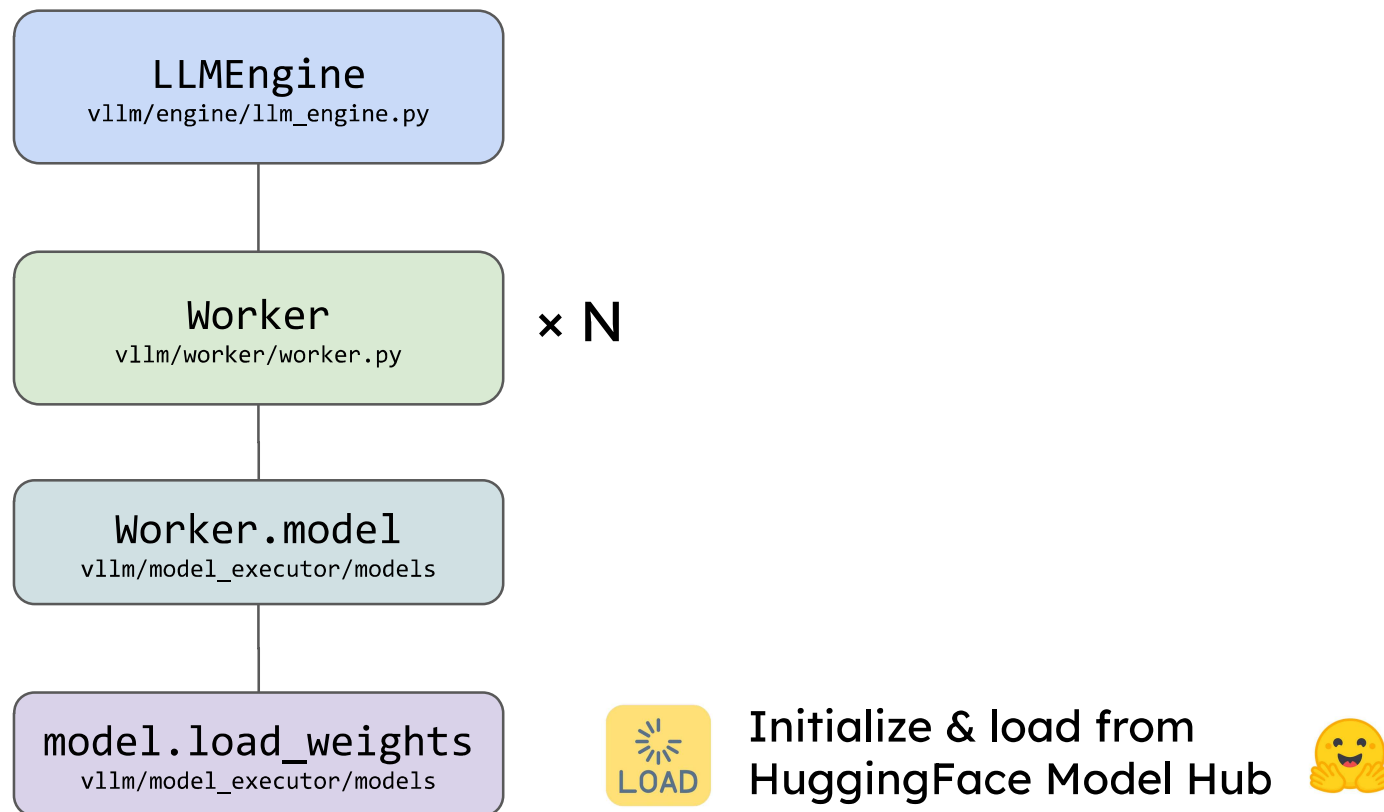
User Interface





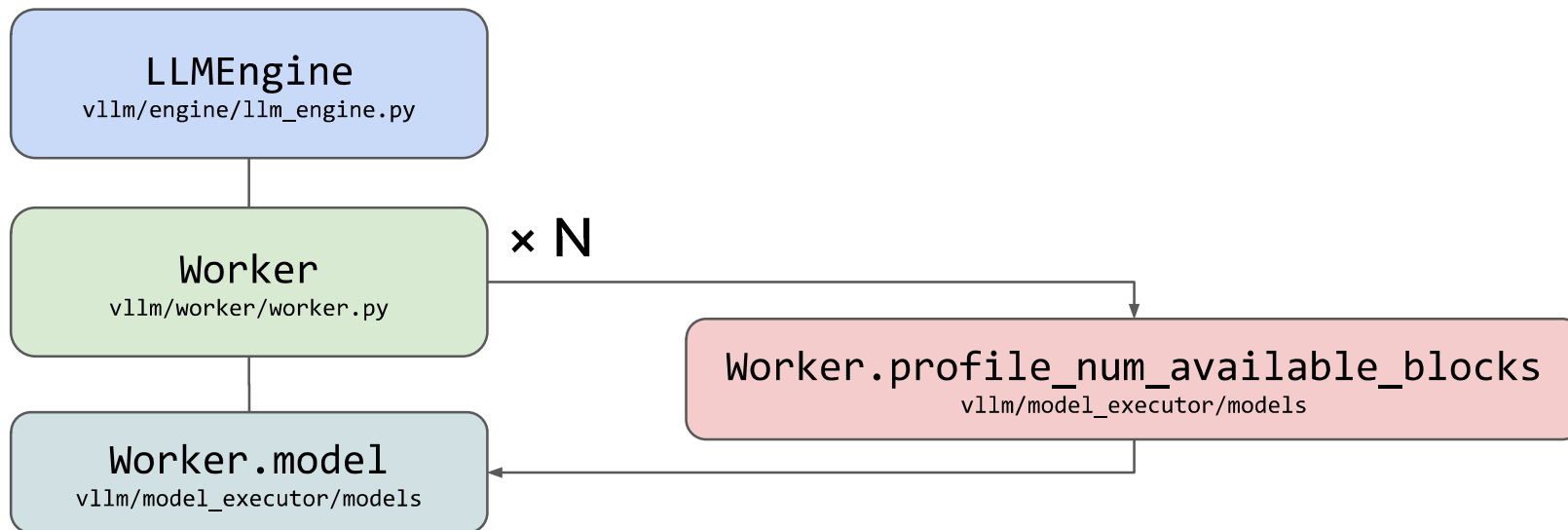
Before serving requests

1. Initialize & load model



Before serving requests

2. *Profile memory usage*

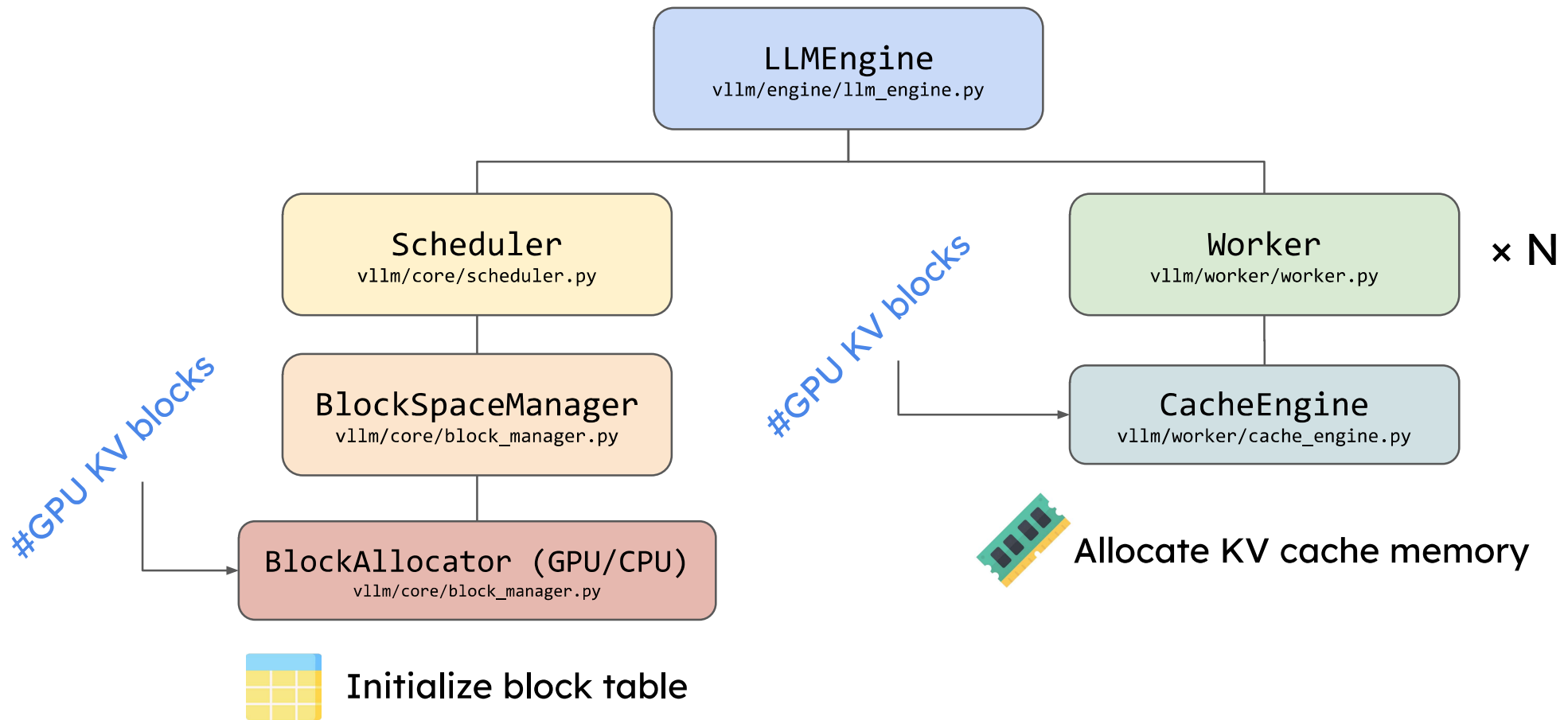


Profile the memory usage with a batch with the max possible #tokens.

#GPU KV blocks = remaining memory / block byte size

Before serving requests

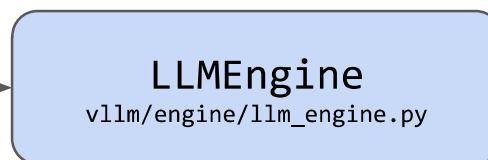
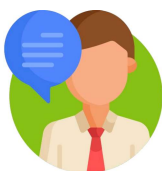
3. Pre-allocate KV Blocks



When requests arrive

Request prompt:

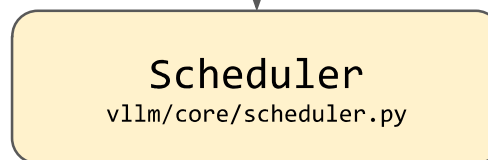
“The future of Artificial Intelligence”



1. Tokenization:

[1, 450, 5434, 310, 3012, 928, 616, 3159, 28286]

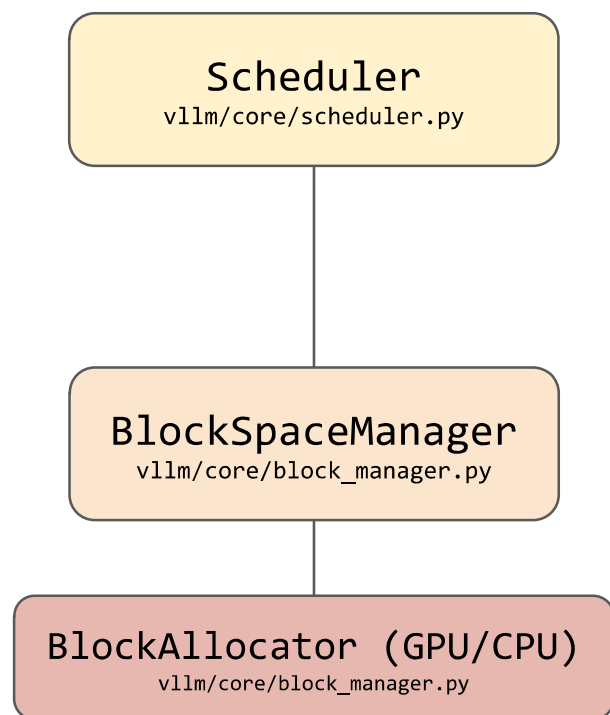
2. Add to the scheduler's **waiting queue**



- **Waiting** request queue
- **Running** request queue
- **Swapped** request queue



At every step, the scheduler



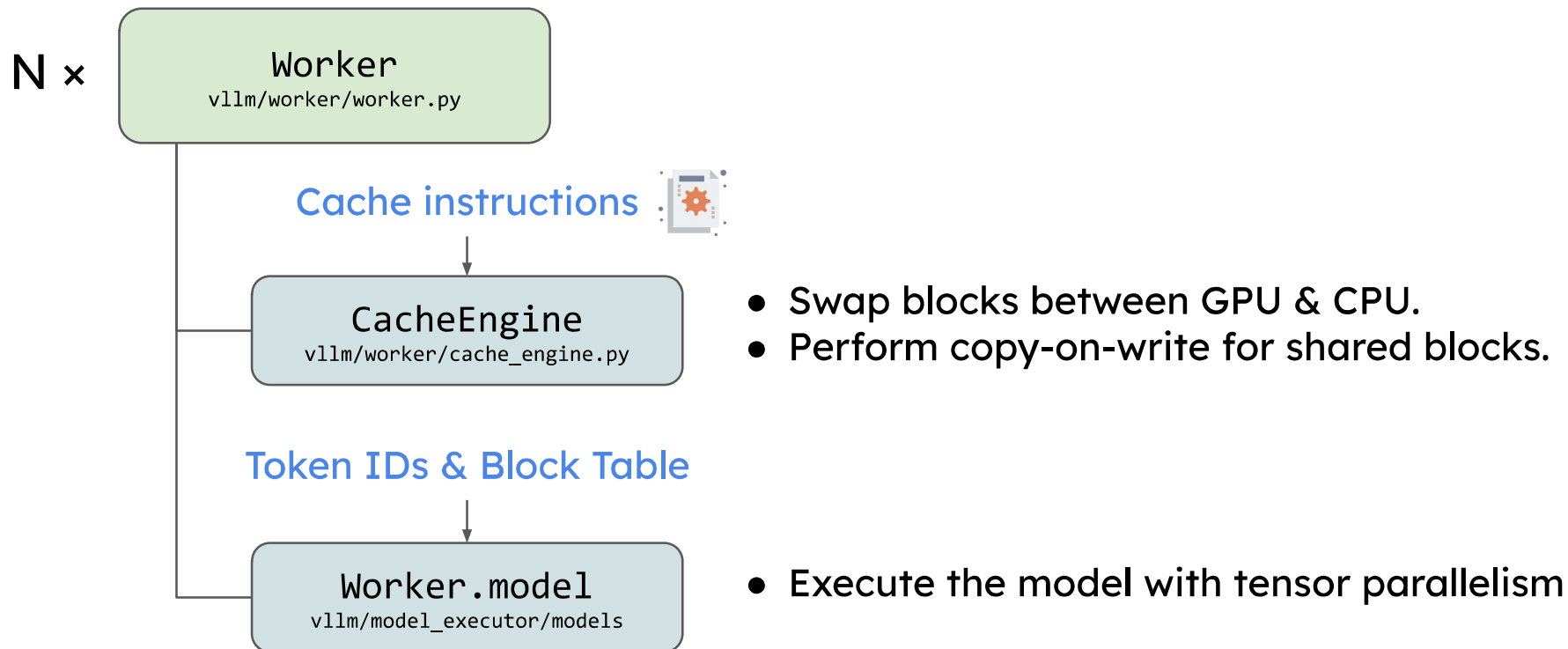
Decide the set of requests to run at the current step.

- When there are free KV block memory
waiting → **running**
- When no KV block memory available for new tokens:
Swapping: **running** → **swapped**
Recomputation: **running** → **waiting**

- Allocate space for new KV Cache.
 - Handle block sharing.
 - Swap/delete preempted KV blocks.
- **Cache instructions & Block table**

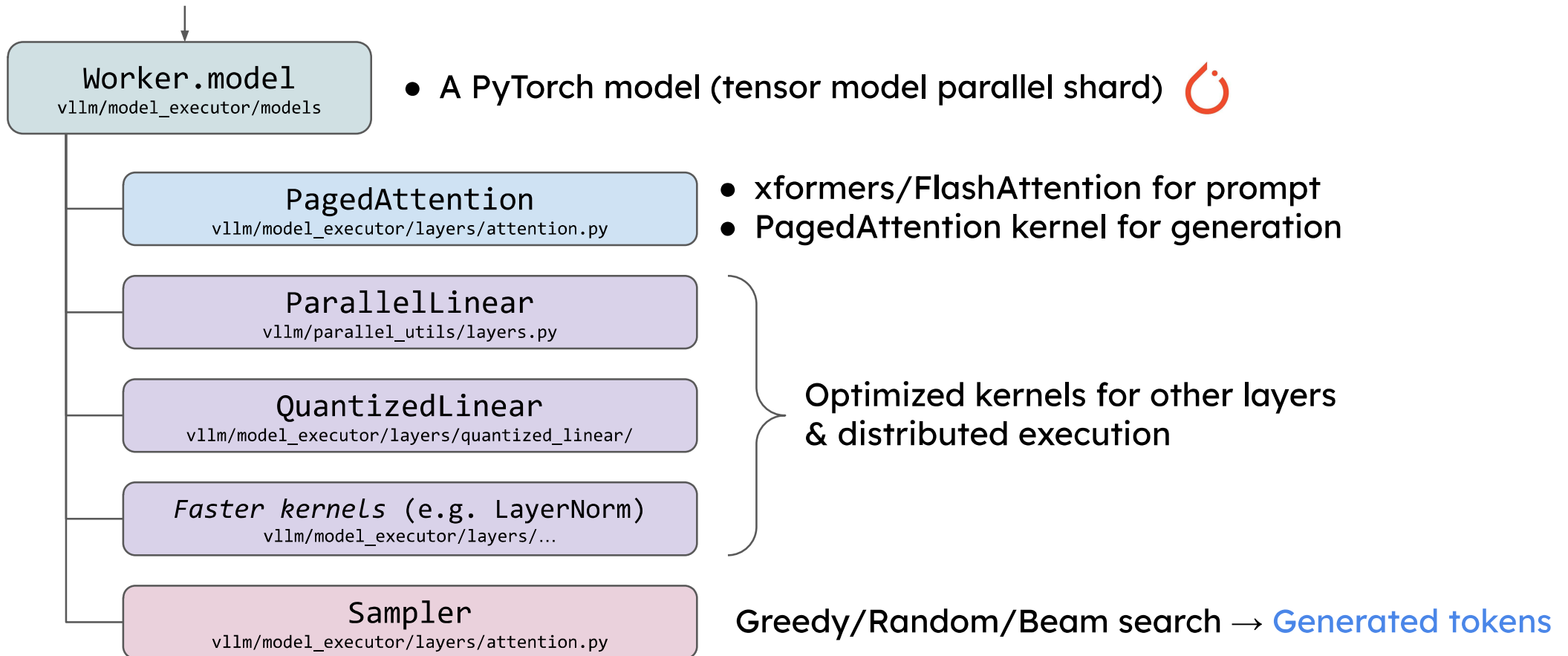


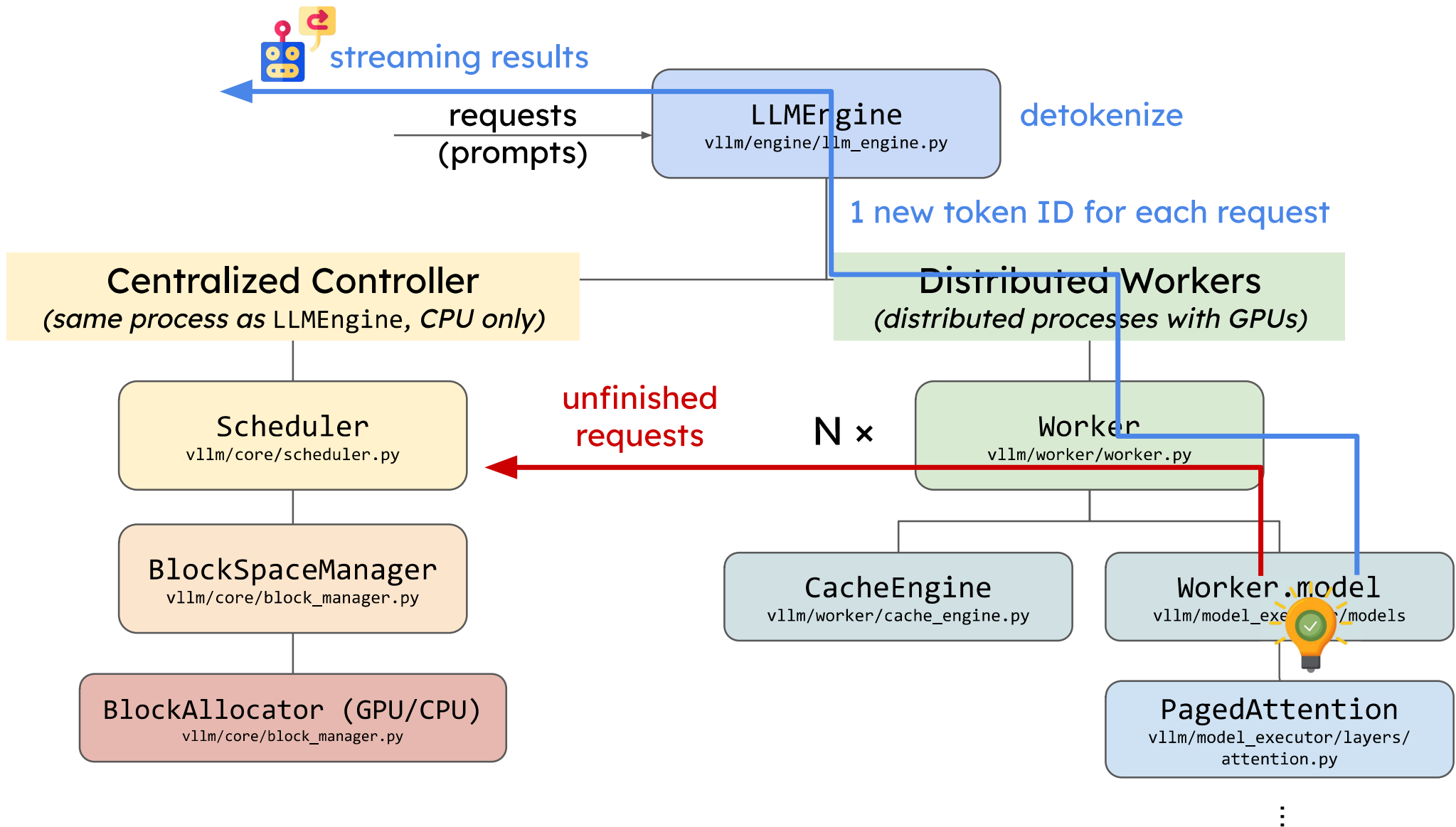
At every step, the worker



At every step, the model

Token IDs & Block Table





Summary

1. Core component: **LLMEngine**
2. Centralized controller → Distributed workers
3. Scheduler prepares the requests at each step
4. Workers run the model with **PagedAttention**

vLLM Recent Updates

Woosuk Kwon



Our journey since the project release

1. Supporting new models
2. Optimizing performance
3. Ensuring correctness & robustness
4. Improving usability
5. Code gardening & refactoring

Our journey since the project release

1. Supporting new models

2. Optimizing performance

3. Ensuring correctness & robustness

4. Improving usability

5. Code gardening & refactoring

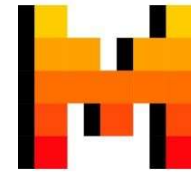
Supported models



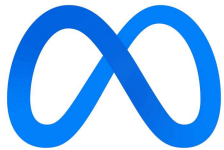
LLaMA/Vicuna/
LLaMA-2/CodeLlama



GPT-J/NeoX



Mistral



OPT



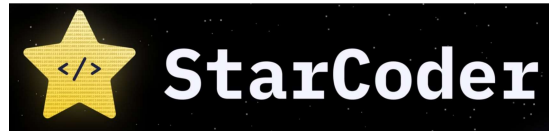
GPT



Falcon



BLOOM



StarCoder



MPT

Contributed by MistralAI
Supported from Day 1

+ 4 more architectures

+ RoPE scaling

+ Quantization

Our journey since the project release

1. Supporting new models

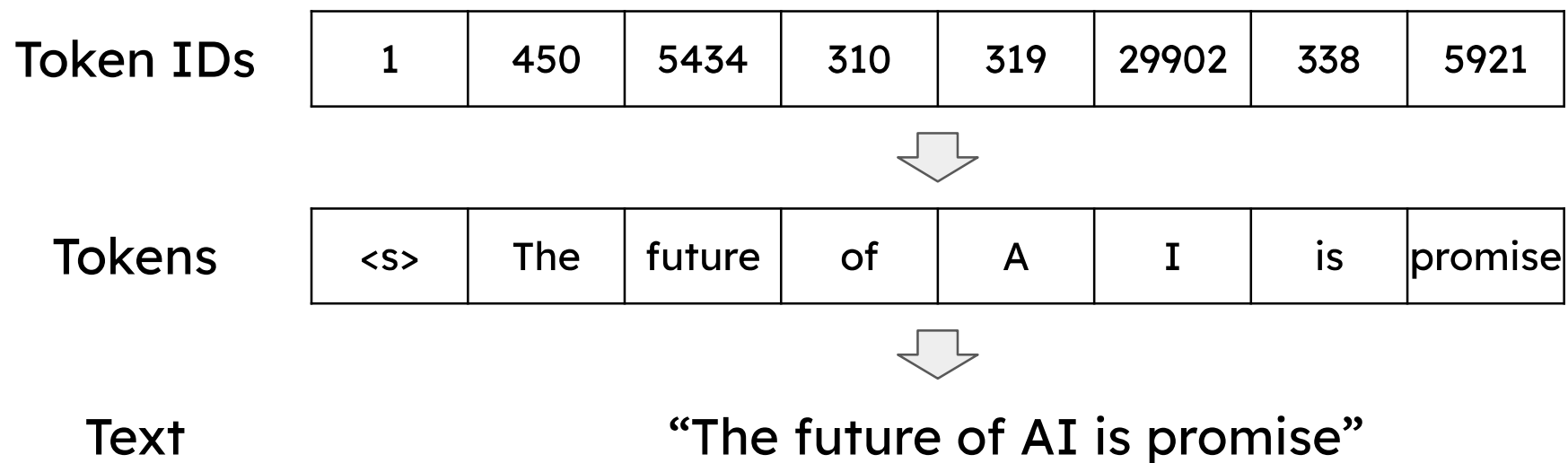
2. Optimizing performance

3. Ensuring correctness & robustness

4. Improving usability

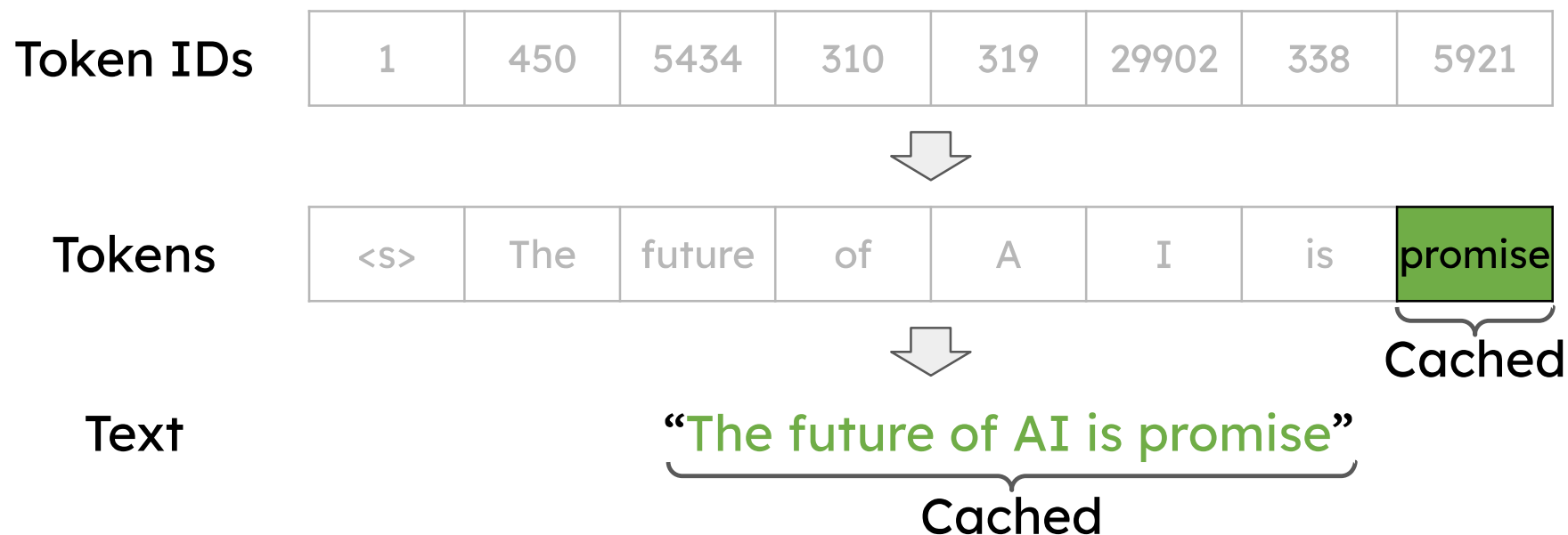
5. Code gardening & refactoring

Optimization (1): Efficient de-tokenization



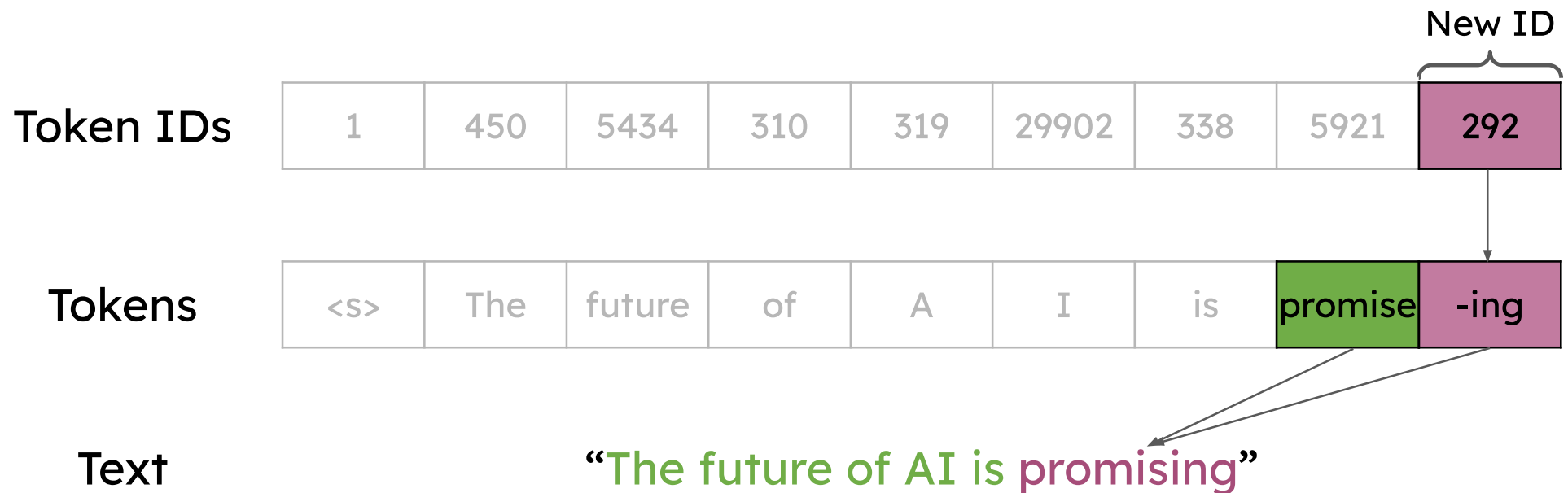
Optimization (1): Efficient de-tokenization

Reduces CPU overhead by **caching** and **incremental updates**



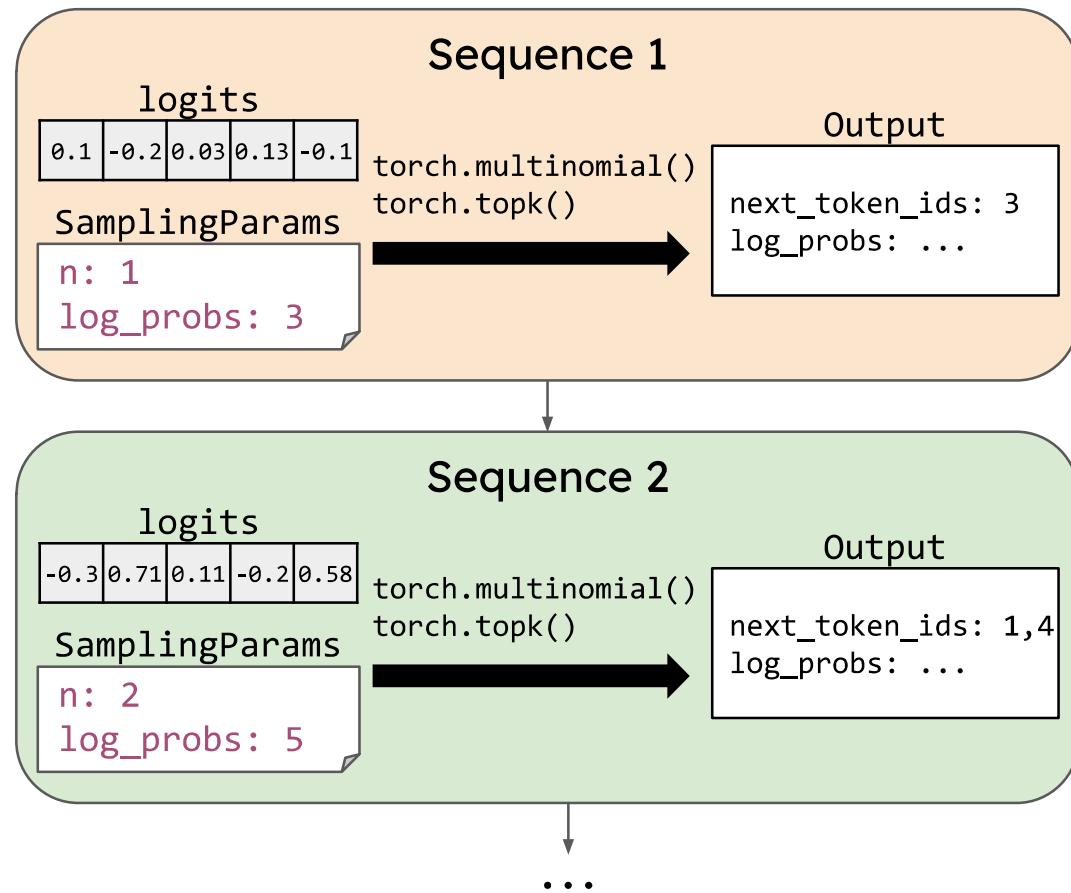
Optimization (1): Efficient de-tokenization

Reduces CPU overhead by **caching** and **incremental updates**

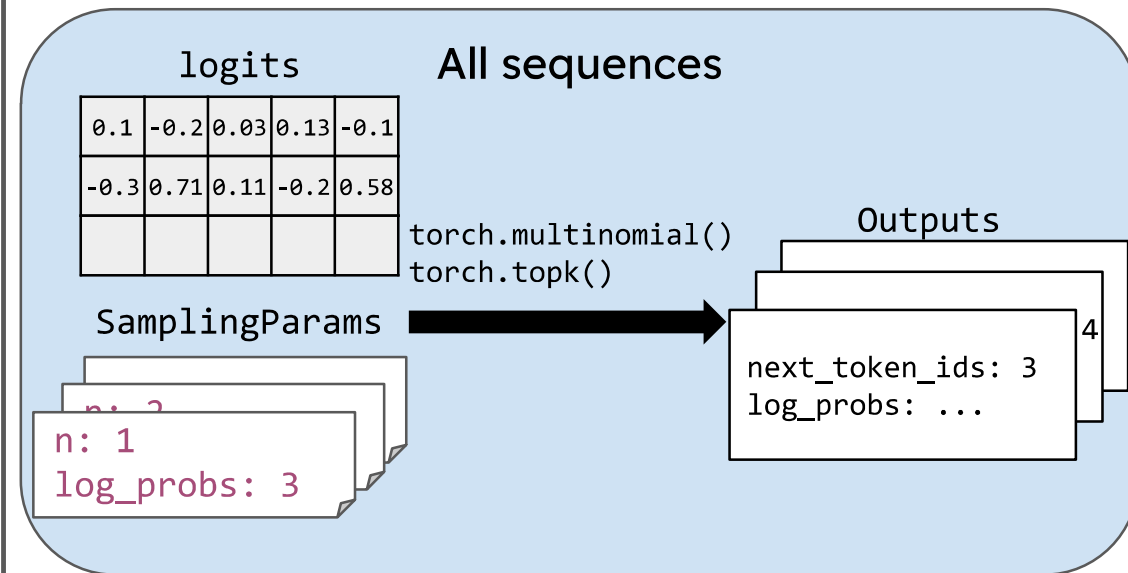


Optimization (2): Vectorized sampler

Iterative (Previous)

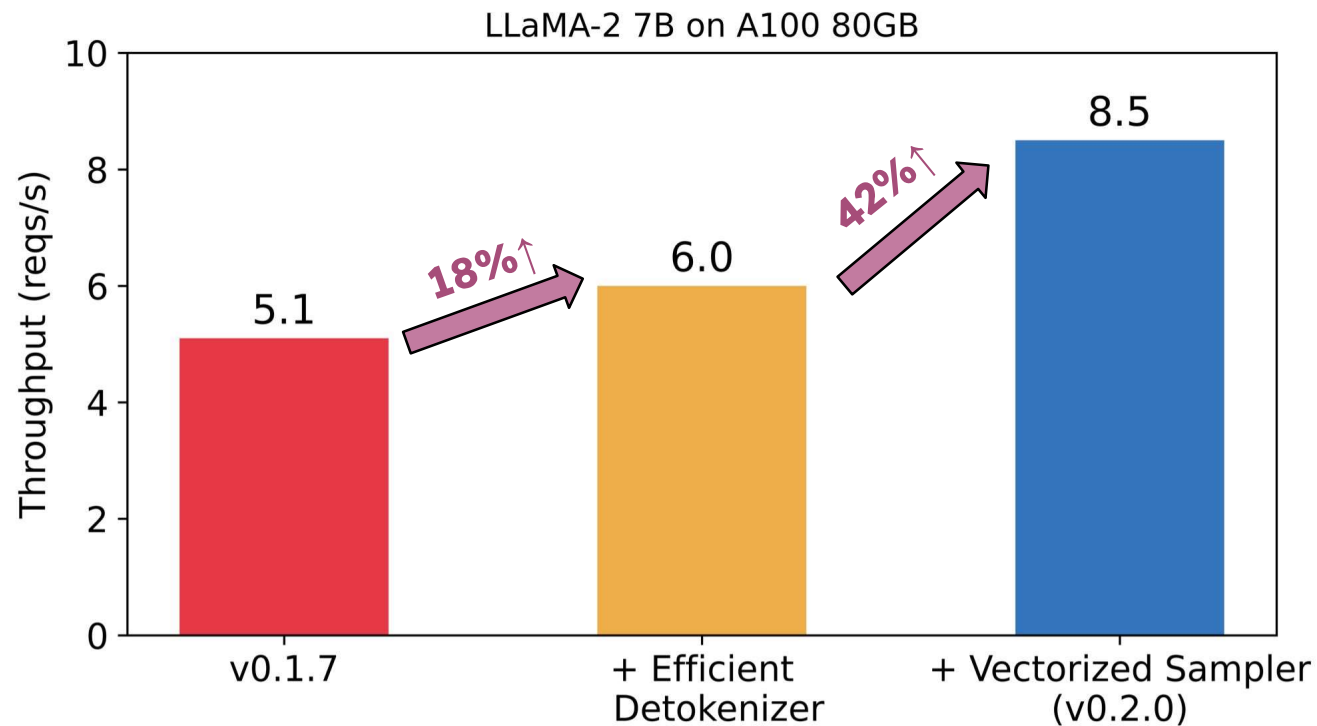


Vectorized (Current)



Performance improvements

v0.2.0: Up to **66%** throughput increase compared to v0.1.7



Our journey since the project release

1. Supporting new models

2. Optimizing performance

3. Ensuring correctness & robustness

4. Improving usability

5. Code gardening & refactoring

Ensuring correctness is hard

- Because of the custom CUDA kernels, vLLM cannot always ensure **bit-wise** correctness

Ref. logits:

-0.88	0.10	...	0.13	-0.06
-------	------	-----	------	-------



vLLM logits:

-0.85	0.10	...	0.17	-0.09
-------	------	-----	------	-------

- As a result, vLLM can sometimes generate **different** output compared to reference implementations

Ref. output: “The future of AI is **unpredictable but full of potential.**”



vLLM output: “The future of AI is **promising and transformative.**”

Tests for ensuring correctness

1. Unit tests for op-level correctness

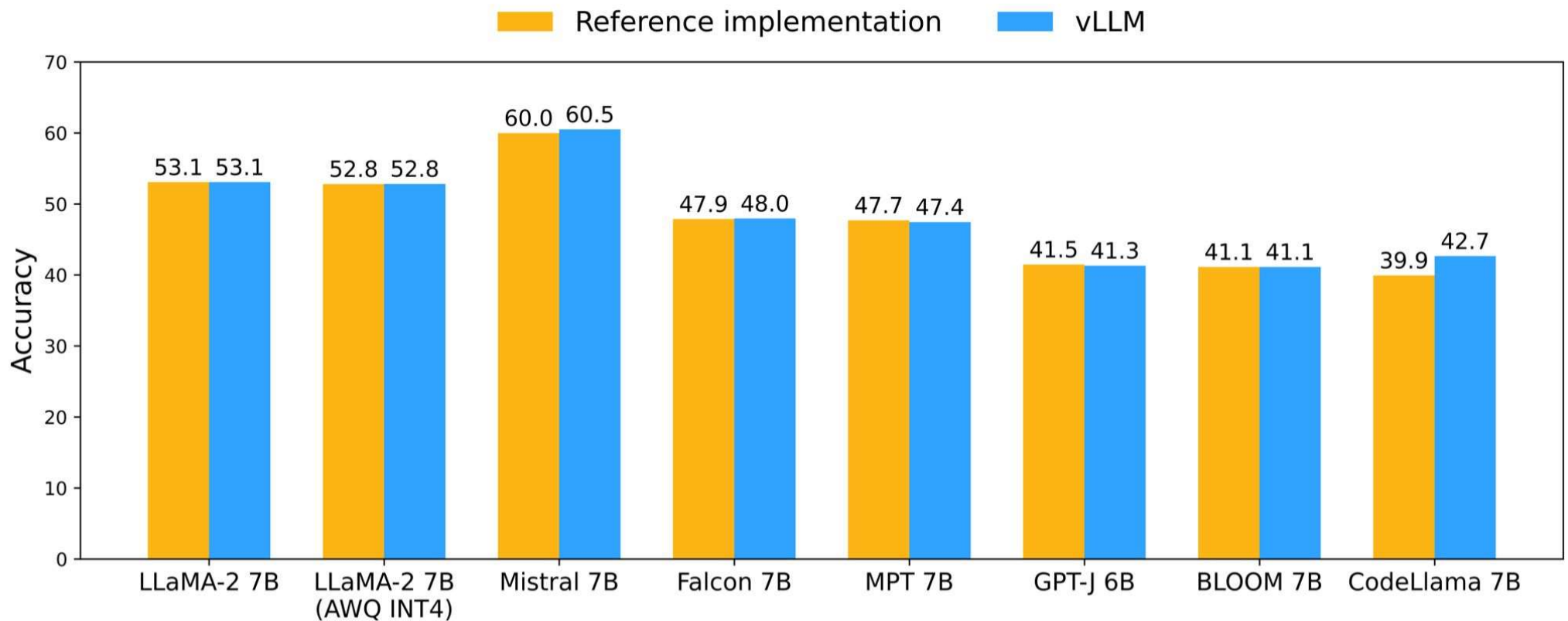
2. End-to-end tests for certain cases

- Ex) argmax sampling, beam search

3. Accuracy tests on standard benchmarks

Accuracy tests

- Benchmark: AI2 Reasoning Challenge (ARC)*



* acc_norm with 25 shots, <https://github.com/EleutherAI/lm-evaluation-harness>

Correctness & robustness

Making AsyncLLMEngine robust

- PR [#880](#), [#943](#), [#969](#), [#988](#), [#1059](#), [#1102](#)
- Special thanks to Antoni @ Anyscale

Making beam search algorithm compatible with HF Transformers

- PR [#646](#), [#847](#)

Bug fixes for corner cases

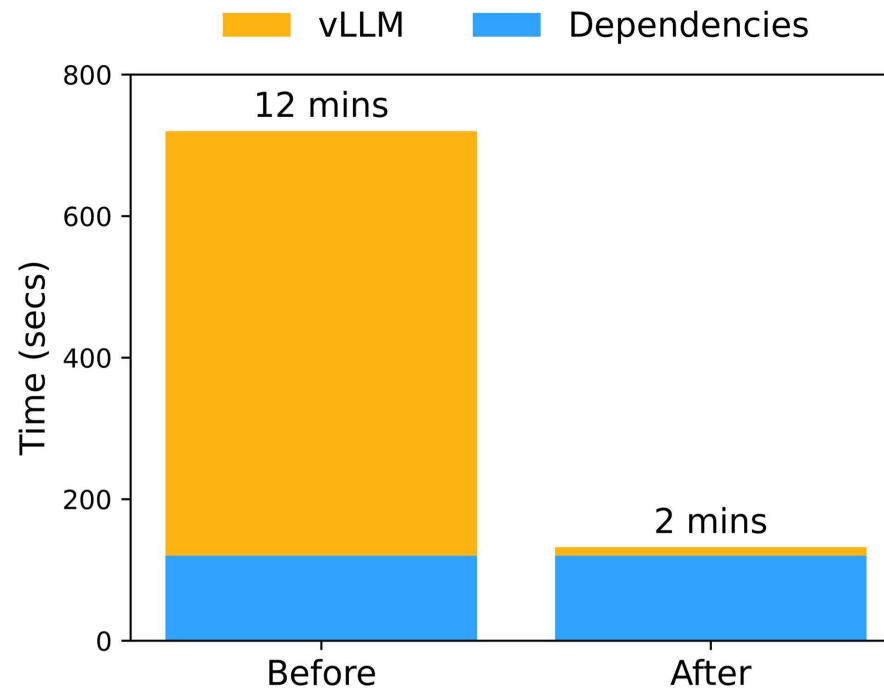
- PR [#936](#), [#1004](#), [#1154](#), [#1241](#)

Our journey since the project release

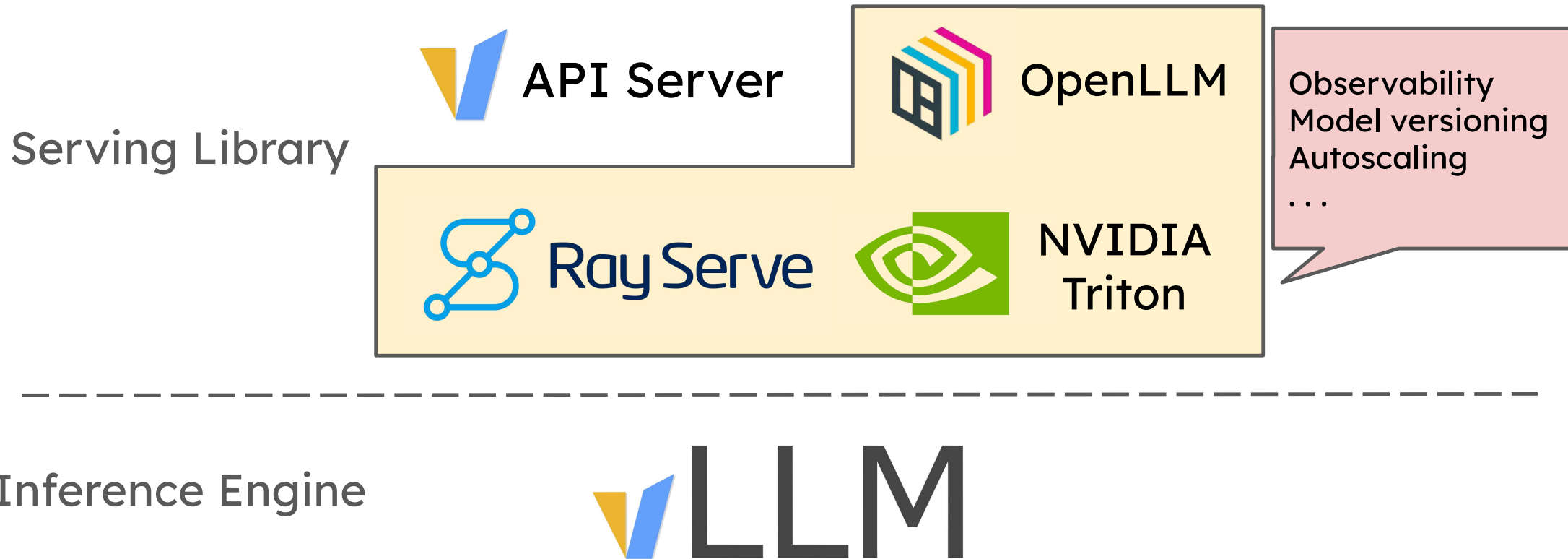
1. Supporting new models
2. Optimizing performance
3. Ensuring correctness & robustness
4. Improving usability
5. Code gardening & refactoring

Pre-built CUDA binaries (since v0.1.4)

- No need for compilation
- Fast installation
 - **12 secs** for installing vLLM
 - **2 mins** for end-to-end installation



Integration with serving libraries



Our journey since the project release

1. Supporting new models
2. Optimizing performance
3. Ensuring correctness & robustness
4. Improving usability
5. Code gardening & refactoring

Maintaining simple and clean code

Goal: Making vLLM easy to understand, hack, and contribute

- High standard for code complexity
- Refactoring
 - Scheduler (PR [#658](#), [#1251](#))
 - Tensor parallelism (PR [#1181](#))
 - Quantization (PR [#926](#), [#1032](#))
 - Ray utils (PR [#397](#))
 - AsyncLLMEngine (PR [#880](#))

vLLM Roadmap

Woosuk Kwon & Zhuohan Li



vLLM will remain open-source



Apache 2.0 License

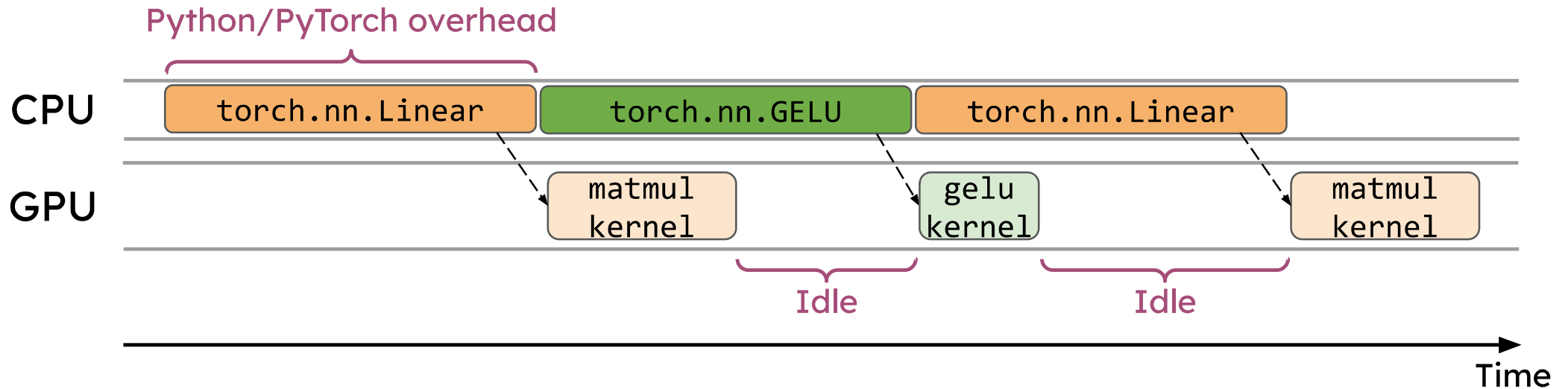
Future Roadmap (Overview)

Optimizing latency

Better quantization support

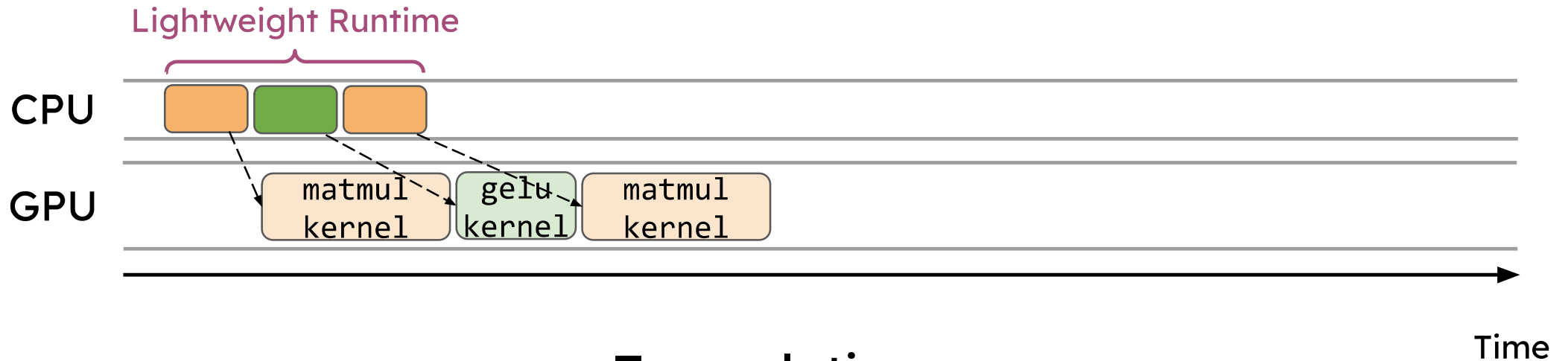
CUDA-level optimizations

Optimizing latency (1): Lightweight runtime



Python/PyTorch overhead takes up to **50%** of overall latency

Optimizing latency (1): Lightweight runtime



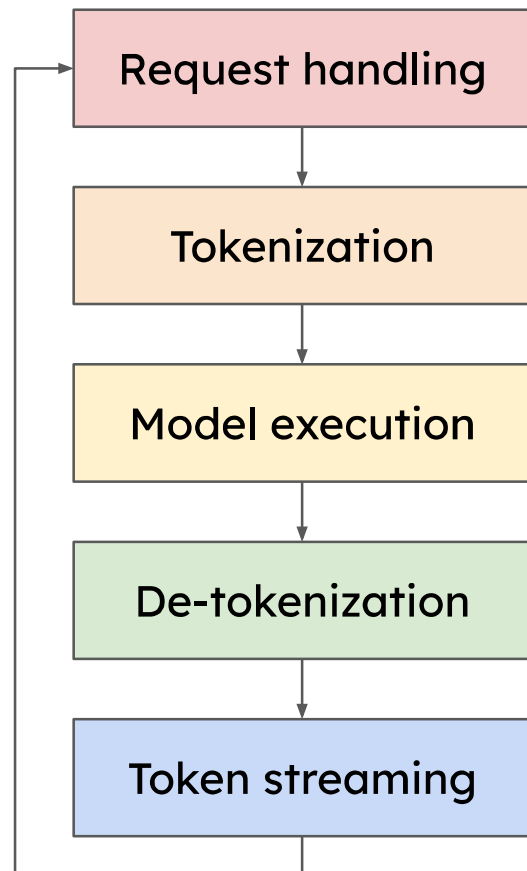
Two solutions



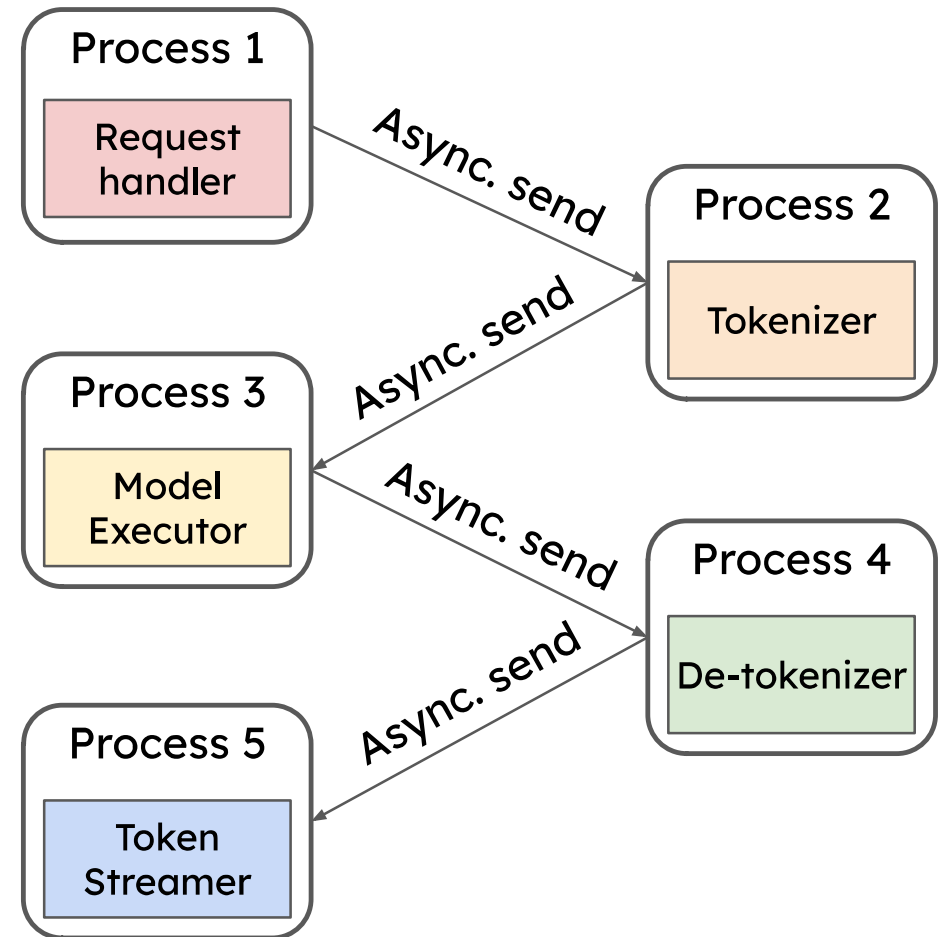
Optimizing latency (2): Multi-process architecture

Single-process architecture

(Current)

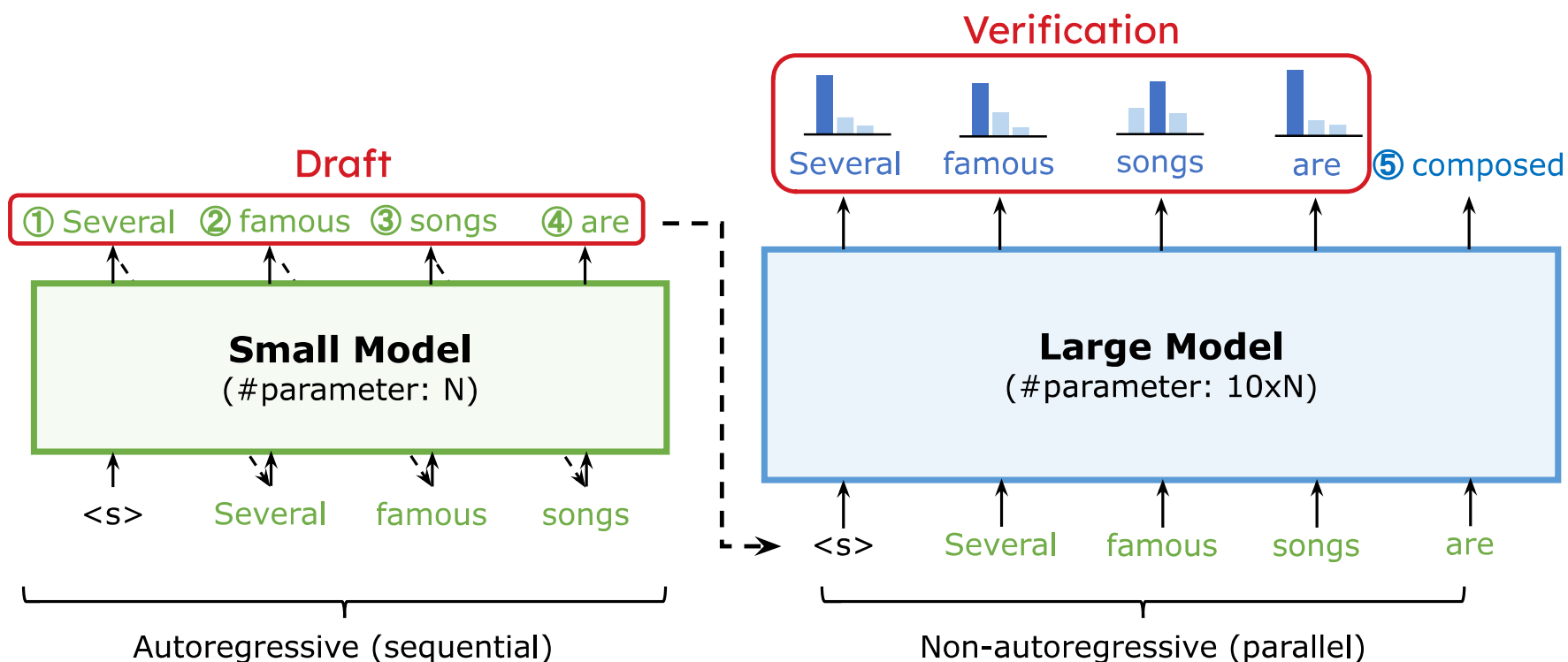


Multi-process architecture



Optimizing latency (3): Speculative decoding

Small model writes a **draft** → Large model **verifies** it



* Figure adapted from Kim et al., "Speculative Decoding with Big Little Decoder" (NeurIPS 2023)

Better quantization support

Step 1: Develop a general abstraction for diverse quantization methods.



Step 2: Implement diverse quantization methods.

- [WIP] GPTQ, SmoothQuant, SqueezeLLM
- [TODO] GGML, GGUF, ...



Step 3: Optimize the performance of quantized ops.

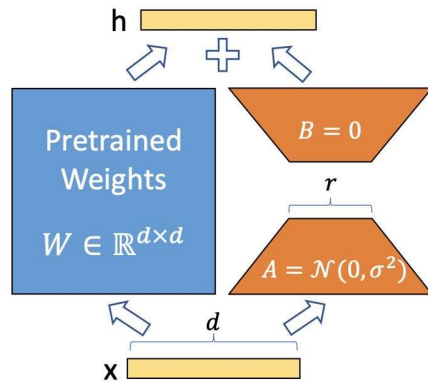
CUDA-level optimization: PagedAttention kernel

PagedAttention takes **20-40%** of overall running time

Optimization 1: Better work partitioning
(leveraging sequence-level parallelism)

Optimization 2: Efficient MQA/GQA support

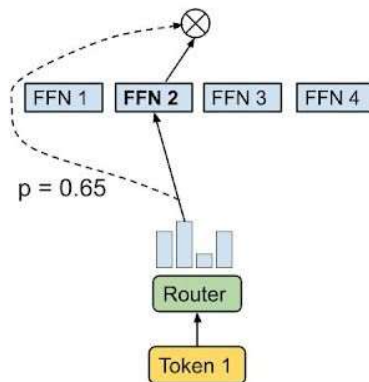
Second-priority items



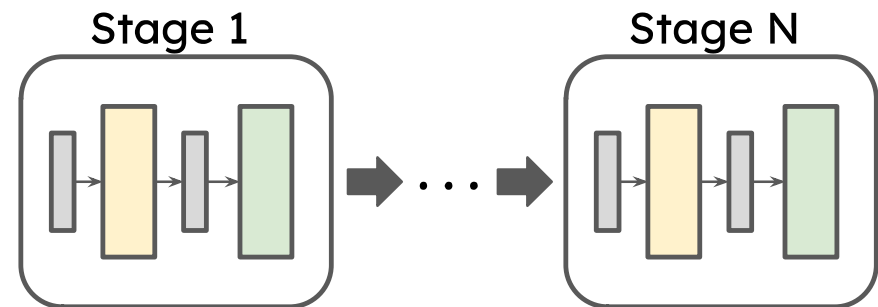
(Efficient) LoRA support*

AMD
ROCm

AMD GPU support



Mixture-of-Experts



Pipeline parallelism

* Figure adapted from Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models" (ICLR 2022)

Q&A on future roadmap (5 min)

First-Priority Features

- Latency optimization
 - Lightweight runtime
 - Multi-process architecture
 - Speculative decoding
- Better quantization support
- CUDA-level optimization

Second-Priority Features

- (Efficient) LoRA support
- AMD support
- Mixture-of-experts
- Pipeline parallelism

vLLM Networking Hour!



<https://github.com/vllm-project/vllm>



<https://vllm.ai>



<https://arxiv.org/abs/2309.06180>



<https://discord.gg/jz7wJKhh6g>



@woosuk_k



@zhuohan123

**Please fill out our survey
before you leave:**

